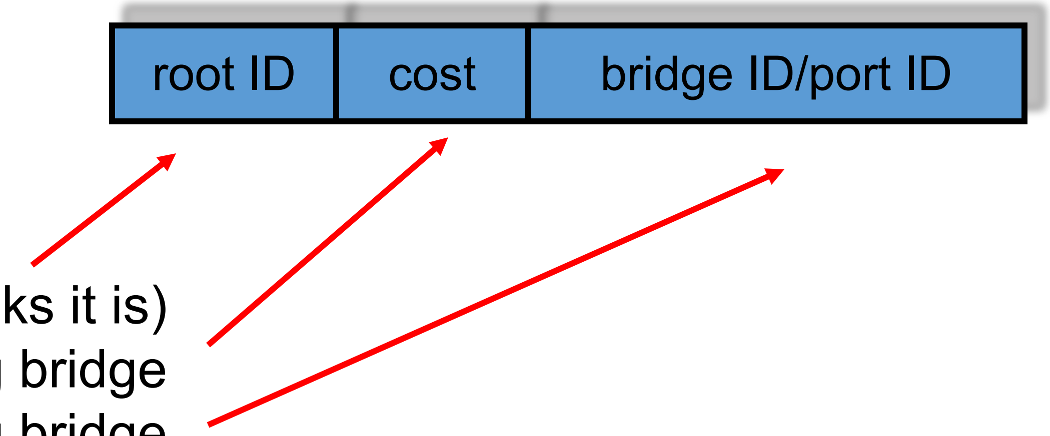
Steps of Spanning Tree Algorithm

1. **Determine the root bridge**
2. **Determine the root port on all other bridges**
3. **Determine the designated port on each LAN**

Each bridge sends out BPDUs that contain the following information:



root bridge (what the sender thinks it is)
root path cost for sending bridge
Identifies sending bridge

Ordering of Messages

- We can order BPDU messages with the following ordering relation “**<<**“:

M1	ID R1	C1	ID B1	<	ID R2	C2	ID B2	**M2**

If (R1 < R2)

M1<< M2

elseif ((R1 == R2) and (C1 < C2))

M1 << M2

elseif ((R1 == R2) and (C1 == C2) and (B1 < B2))

M1 << M2

- If above holds, M1 is dominant and M2 will change its information to match M1
- Else M2 is dominant and M1 changes to follow M2

Determine the Root Bridge

- initially, all bridges assume they are the root bridge.
- each bridge B sends BPDUs of this form on its LANs:

B	0	B

- each bridge looks at the BPDUs received on all its ports and its own transmitted BPDUs.
- root bridge, at any point in time, is the **smallest received root ID** that has been received so far.
    - Whenever a smaller bridge ID arrives, the root field is updated in a bridges BPDU root bridge field
    - Otherwise bridge maintains the current value

Calculate the Root Path Cost - Determine the Root Port

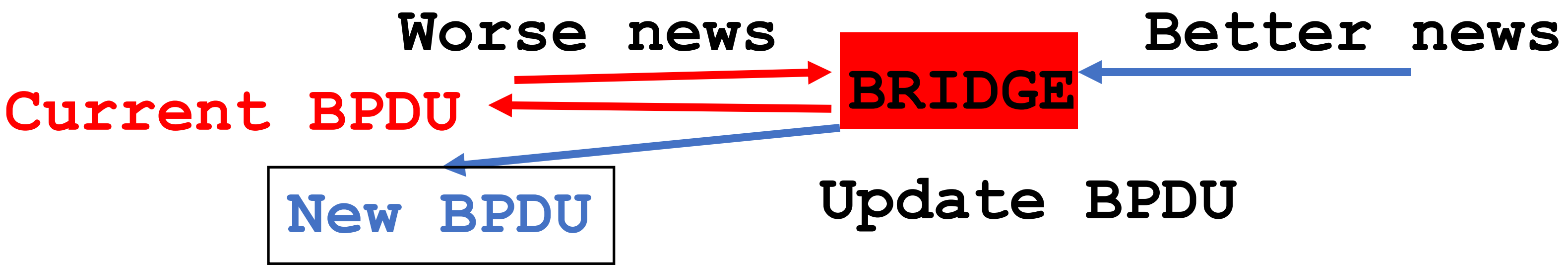
- at some point bridge B has a belief of who the root is, say R.
- bridge B determines the Root Path Cost (Cost) as follows:
  - *if* $B = R$: Cost = 0 (i.e., it is the root)
  - *if* $B \neq R$: Cost = {Lowest Cost found in a received BPDU with R as root} + 1

Updating and Sharing Information

- **B's root port** is the port from which **B** received the lowest cost path to R (in terms of relation "**<<**"). E.g., **port A**
- knowing **R** and **Cost**, **B** can generate a current BPDU with its root port **A**.

R	Cost	B	A

- bridge B will only send its current BPDU to a neighbor if it receives "worse news" on any of its ports from that neighbor.
- it updates its BPDU it if receives "better news" from a neighbor on one of its ports (that port now becomes the root port) and broadcasts an updated "new" BPDU on all its non root ports.

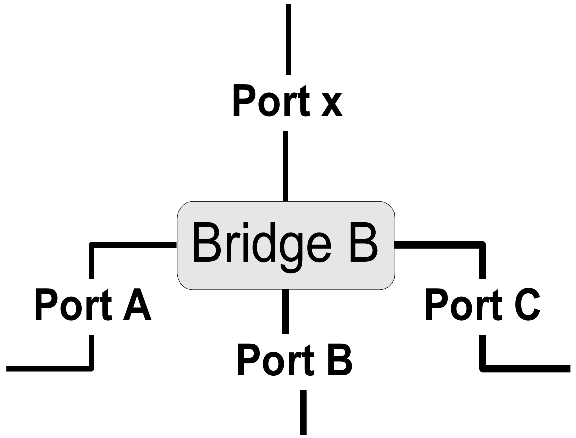


In Summary

- now that B has generated an updated BPDU, it shares the new information on all of its "**other**" ports

R	Cost	B	A

- B will send this BPDU on all of its ports, except port **X**, **if X is port from which it received better news.**
- in this case, B also assumes that it is the **designated bridge** for the LAN to which port **X** connects.
- and **Port X** is the **root port**.



Example

- bridge **B (ID 8)** receives on port "X" the following BPDU from a neighboring bridge (**ID 12**):

R=2	**Cost=3**	12	C

- and **B**'s current BPDU is:

R=2	**Cost=2**	8	A

- because Cost **2** << Cost **3**, **B** will broadcast on its port "**X**" its **current** BPDU to let bridge **(ID 12)** know it has a shorter path to the root bridge (**ID 2**) via port **A** (its current **root** port).
- if instead **B**'s current BPDU is:

2	**5**	8	A

- then **B** will broadcast on all its **other ports** (A,B,C) the following BPDU:

2	**3+1=4**	8	**X**

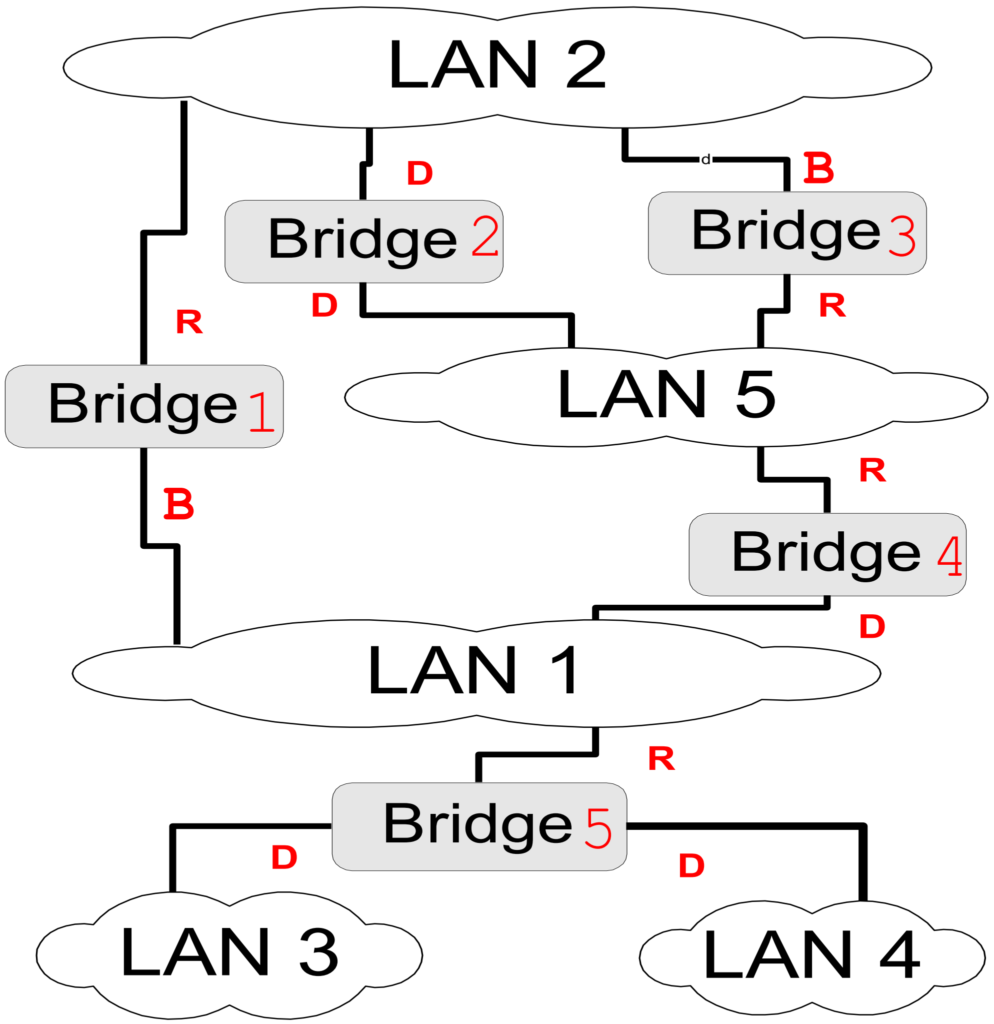
- where "**4**" (3+1) is new cost from Bridge B on port **X** to **root** bridge (**ID 2**) (via bridge **ID 12**). And port **X** is now its **root** port

Selecting the Ports for the Spanning Tree

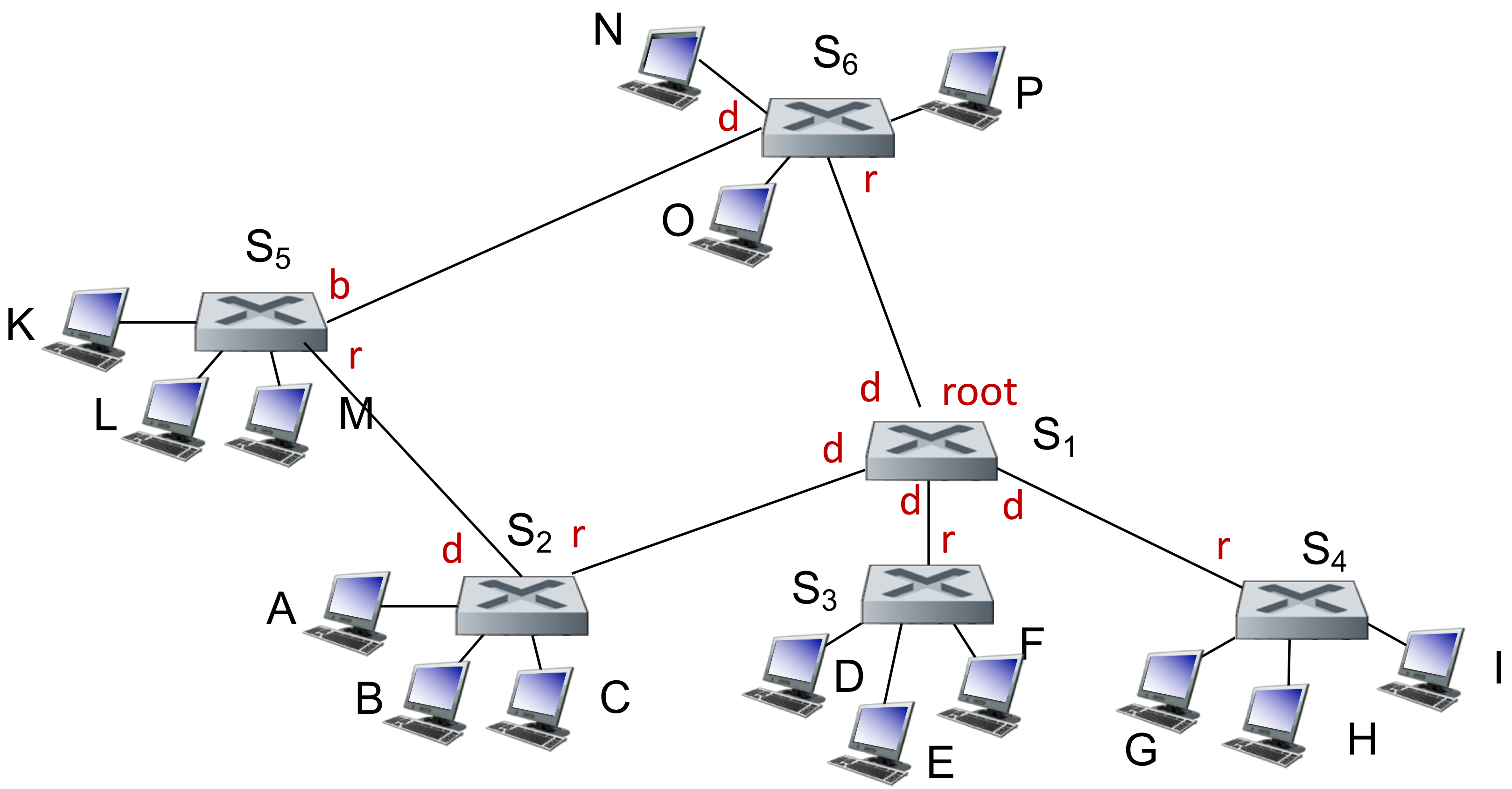
- now that Bridge B has calculated the root, the root path cost, and the designated bridge and port for each LAN.
- **B can decide which ports are in the spanning tree:**
  - B's **root port** is part of the spanning tree (every bridge has to have a root port)
  - The ports on **all** the LANs for which B is the **designated** bridge are part of the spanning tree (**designated ports**).
- B's ports that are in the spanning tree will forward packets **(=forwarding state)**
- B's ports that are not in the spanning tree will not forward packets **(=blocking state)**
- note that a bridge may not be the designated bridge on any LAN. As such, **all its ports**, other than the **root port** will be **blocked**.

Spanning Tree example – what is the spanning tree?

- consider the network on the right.
- assume that the bridges have calculated the designated ports (**D**), the blocked ports (**B**) and the root ports (**R**) as indicated.
- what is the spanning tree?

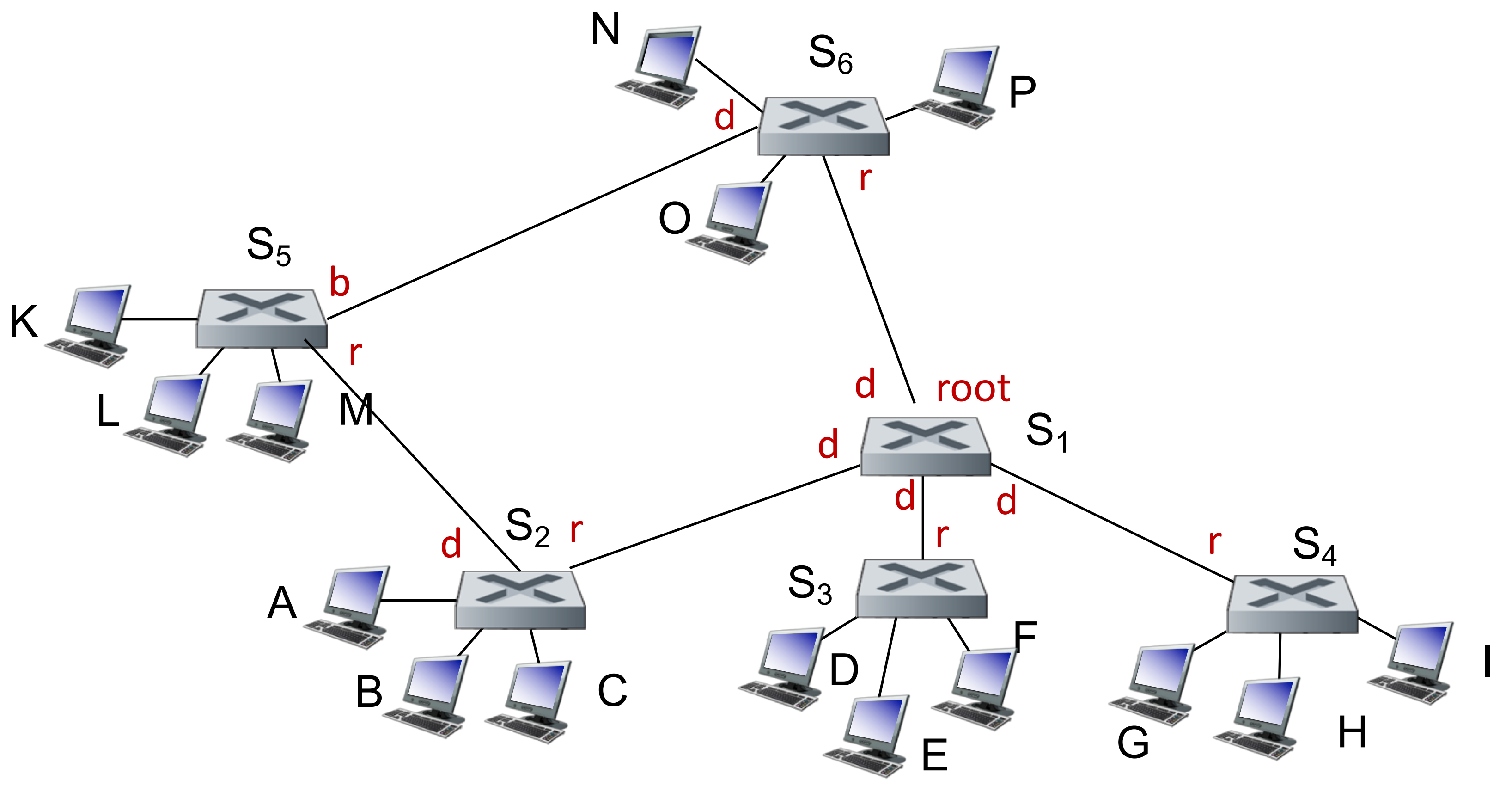


Spanning Tree example – what is the spanning tree?



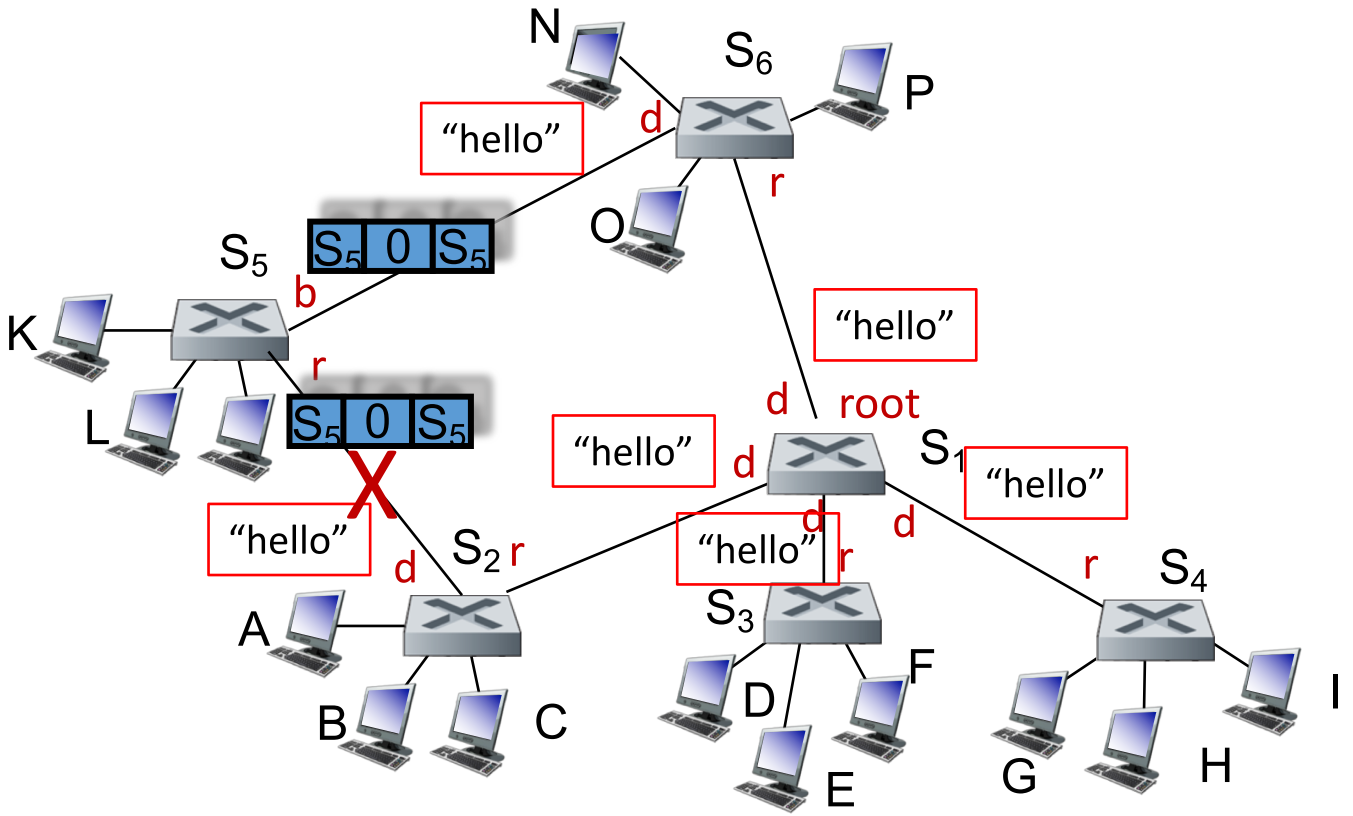
Health of the Spanning Tree

- consider the network on the right - if link between S_5 and S_2 goes down or S_2 fails -> tree is damaged
- S_5 is now isolated and cannot send to, or receive traffic from, the rest of the tree
- using "hello" messages, the switches will notice the failure of a component and re-calculate the spanning tree



Detecting and Recovering from Failures

- *hello* messages are generated periodically by the root switch, which sends them out on each link to the next level of switches
- every switch that receives a *hello* message, replaces the bridge ID with its own, and passes it on to the next level of switches
- all switches receive the *hello* messages on their root ports, and forward them via their designated ports
- if a switch does not receive a hello message within a certain period of time (fixed interval), it starts a "recovery" timer that is 3x the *hello* message interval
- if that timer expires, it will assume the tree is broken and initiate the "spanning tree" process
- when a problem is detected by a switch (i.e., timer expired), it resets all its ports and initiates the spanning tree algorithm by sending out a BPDU declaring itself as Root.



S_5	0	S_5